

VBT: A Queryable Compressed 4D Backend for Animated Volume Assets

ANONYMOUS AUTHOR(S)

SUBMISSION ID: 1937

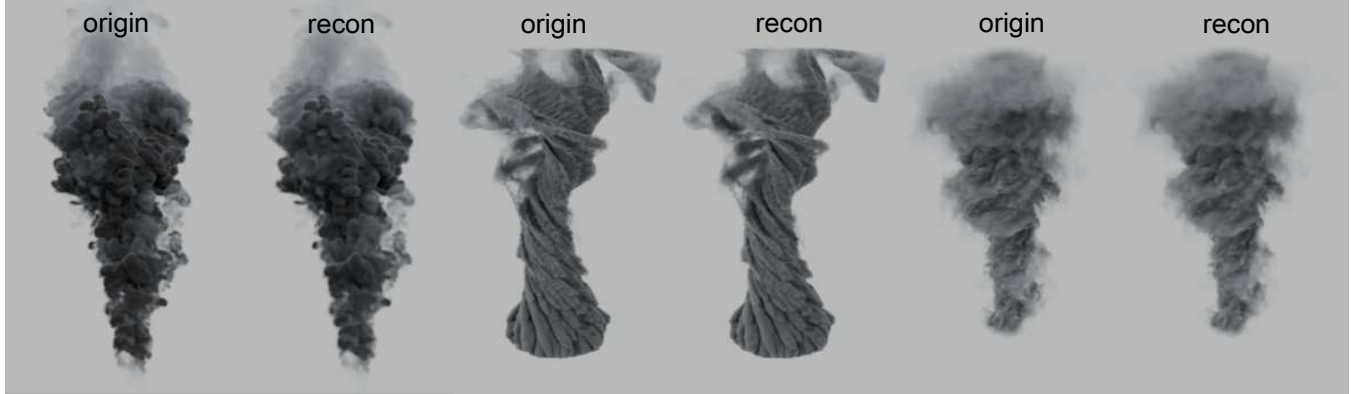


Fig. 1. VBT turns animated smoke/fire caches into compressed .vbtp 4D volume assets that can be loaded and sampled directly by a native Cycles prototype. The resident asset is rendered through CPU and CUDA paths without OpenVDB proxy conversion, producing smoke/fire results from compressed-domain samples rather than materialized dense frames.

Animated smoke, fire, and fog caches are commonly stored as per-frame volume assets, which makes long sequences expensive to store, stream, and render interactively. We present VBT, a compressed 4D volume asset backend that stores an entire time-varying scalar volume in a resident packed representation and supports direct (x, y, z, t) sampling without materializing dense frames. VBT uses a flat offset table, mode-tagged block payloads, and bounded mode-dependent GPU dispatch. For rendering-oriented volumes, a temporal-first field-adaptive encoder preserves visible transient density events before spatial residual organization. We integrate VBT into a native Cycles prototype that loads .vbtp assets directly and renders them on CPU and CUDA paths, bypassing OpenVDB proxy conversion. Across four smoke/fire sequences, VBT reduces storage by 25.9–60.3 $\times$ relative to dense RAW sequences while preserving visual structure under fixed Blender/Cycles settings. Compared with conventional per-frame OpenVDB caches, VBT stores the sequence as a single temporally compressed .vbtp asset: across four smoke/fire sequences, it is 25.9–60.3 $\times$ smaller than dense RAW sequences and about 1.8–5.1 $\times$ smaller than the corresponding source OpenVDB assets, while supporting direct native Cycles sampling from the resident compressed representation. Auxiliary dense scientific fields are used only to stress-test the same direct-query backend beyond sparse rendering assets. These results position VBT as a compressed 4D runtime asset backend for animated volumes rather than a replacement for per-frame sparse volume layouts or scientific codecs optimized solely for RMSE or error bounds.

CCS Concepts: • **Computing methodologies** $\rightarrow$ **Volumetric models; Volumetric rendering.**

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM. ACM 1557-7368/2026/5-ART

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

Additional Key Words and Phrases: animated volume assets, volumetric compression, compressed-domain rendering, random access, GPU query

ACM Reference Format:

Anonymous Author(s). 2026. VBT: A Queryable Compressed 4D Backend for Animated Volume Assets. *ACM Trans. Graph.* 1, 1 (May 2026), 10 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Animated volumetric effects such as smoke, fire, explosions, and fog are central assets in film, games, and DCC workflows [Drebin et al. 1988; Levoy 1988; Max 1995]. These effects are commonly cached as sequences of sparse volume frames, for example OpenVDB assets, and then loaded, previewed, sampled, and rendered by downstream tools. This per-frame organization is practical, but it does not directly represent the sequence as a compressed 4D asset: temporal coherence is left outside the runtime representation, and rendering systems often need to stream or cache multiple frames to evaluate time-varying density. We ask whether an animated volume sequence can instead be represented as a single compressed 4D runtime asset: compact on disk, resident on the GPU, and directly sampled by a renderer at arbitrary (x, y, z, t) coordinates. A practical animated-volume backend for this goal must satisfy several properties simultaneously:

- **Direct renderer sampling.** A renderer should be able to evaluate density from the compressed asset without first materializing dense frames or converting through a proxy volume.
- **Random 4D access.** Any sample at an arbitrary (x, y, z, t) coordinate should be recoverable without decompressing neighboring blocks or adjacent time steps.

- **Bounded decode.** The cost of decoding a block must be predictable and small, enabling GPU dispatch with explicit mode-dependent work.
- **High compression.** Storage must be reduced substantially relative to raw floating-point sequences while retaining visible transient density events.
- **Local behavior.** Dense physical-query workloads are sensitive to isolated local failures, so the same backend should expose predictable local-error behavior rather than optimizing only average distortion.

Existing approaches satisfy subsets of these properties but not all of them together. Sparse graphics structures such as VDB [Museth 2013] and NanoVDB [Museth 2021] support efficient sparse traversal and GPU-friendly per-frame rendering, but they do not by themselves encode a smoke/fire sequence as a temporally compressed 4D asset with direct (x, y, z, t) sampling. Neural representations [Lu et al. 2021; Mildenhall et al. 2020; Müller et al. 2022] achieve impressive compression ratios but optimize a network per scene, provide no explicit block layout, and do not support $O(1)$ packed query. Lossy scientific compressors such as zfp [Lindstrom 2014], the SZ family [Di and Cappello 2016; Liang et al. 2023], and MGARD [Gong et al. 2024; Liang et al. 2022] achieve strong rate-distortion on dense fields but target a different design point: offline numerical reconstruction or frame-local block access rather than a renderer-facing resident 4D asset backend. None of these lines of work directly addresses the combined requirement of temporal compression, direct compressed-domain random access, bounded per-block decode, and native rendering from one packed animated-volume representation.

We present VBT, a queryable compressed 4D backend for animated volume assets. Unlike conventional file-compression pipelines, VBT is designed from the ground up as a resident packed layout: blocks are addressed by a global offset table, each block carries a self-descriptive mode tag, and decode is a bounded dispatch with no sequential scan. VBT partitions the 4D volume into fixed 8^3 spatial leaves and uses a temporal-first rendering encoder to preserve visible smoke/fire density events before spatial residual organization. The resulting VBTPACK4 asset can be loaded by a native Cycles prototype that samples the resident .vbtp representation directly on CPU and CUDA paths, bypassing OpenVDB proxy conversion. A mode tag in each block header drives a fixed dispatch table on both CPU and GPU, keeping decode cost bounded and explicit. We additionally evaluate dense scientific turbulence fields as stress tests for the same queryable backend, not as the primary application domain and not as a claim of scientific-compression Pareto dominance.

Our contributions are:

- A compressed 4D animated-volume asset backend that stores smoke/fire sequences in a packed .vbtp representation with direct (x, y, z, t) access.
- A temporal-first rendering encoder that preserves visible transient density events in animated smoke/fire volumes while writing into the same packed backend.
- A native Cycles prototype that loads resident .vbtp volumes directly and renders them through CPU and CUDA paths without OpenVDB proxy conversion.

- Auxiliary dense-field stress tests on JHTDB [Li et al. 2008] cutouts showing tail-aware local-error behavior at matched zfp operating points, while explicitly not claiming Pareto dominance over scientific codecs.

2 Related Work

Animated volume assets and DCC rendering. DCC caches commonly store smoke/fire effects as sparse per-frame volumes. OpenVDB [Museth 2013] and NanoVDB [Museth 2021] provide production-proven layouts and GPU-friendly traversal for this setting. They are natural cache representations, but they do not by themselves encode a sequence as a temporally compressed 4D asset with direct (x, y, z, t) sampling. Neural volume representations can compress volumetric scenes [Lu et al. 2021; Mildenhall et al. 2020; Müller et al. 2022], but they typically optimize a network per asset and do not expose a simple packed block layout or deterministic mode-tagged dispatch path inside a renderer. VBT targets a different point: a non-neural packed 4D asset representation that can be sampled directly by a native Cycles path.

Lossy scientific compression. Transform-based and prediction-based compressors such as zfp [Lindstrom 2014], the SZ family [Di and Cappello 2016; Liang et al. 2018, 2023], and MGARD [Gong et al. 2024; Liang et al. 2022] provide strong dense rate-distortion baselines for floating-point scientific fields. Earlier lossless and near-lossless approaches include fpzip [Lindstrom and Isenburt 2006]. Tensor decomposition methods such as Tucker-based compression [Ballester-Ripoll and Pajarola 2016] and TTHRESH [Ballester-Ripoll et al. 2020] achieve competitive rate-distortion on smooth fields. Wavelet-based 3D compression [Ihm and Park 1999] pioneered interactive decomposition for large volumes. GPU-accelerated variants [Wu et al. 2023] further improve throughput. A recent survey [Lindstrom et al. 2025] provides a comprehensive overview of lossy techniques for scientific data.

These methods are highly effective for numerical data reduction and offline reconstruction, but they target a different runtime contract from animated-volume rendering. A renderer-facing smoke/fire asset must be sampled repeatedly at arbitrary spatial locations and time values, often while scrubbing or advancing frames, without first materializing dense frames or converting through a proxy representation. Fixed-rate zfp supports block-level random access and can be used for frame-local GPU query benchmarks, but it is not formulated as a native packed 4D animated-volume asset with explicit temporal semantics, mode-tagged renderer dispatch, and direct (x, y, z, t) sampling inside a rendering backend. The SZ family and MGARD are therefore the correct numerical baselines for our auxiliary dense-field stress tests, but they are complementary to the main VBT goal: resident compressed-domain rendering of animated smoke/fire assets.

Sparse and random-access volume structures. GVDB [Hoetzlein 2016] extends sparse voxel databases to the GPU, and GigaVoxels [Crassin et al. 2009] introduced ray-guided streaming for efficient voxel rendering. These systems demonstrate the value of sparse traversal and random-access volume rendering, but they do not provide the packed temporal compression and direct resident 4D query contract targeted here.

Compressed access. Prior work on compressed-domain rendering and GPU sparse sampling has shown the value of bounded decode paths and self-descriptive payloads [Hoang et al. 2021; Laine and Karras 2011; Rodríguez et al. 2014; Schneider and Westermann 2003]. Our design follows this direction but supports multiple residual families under the same packed query interface.

Time-varying volumes. Time-space partitioning trees [Shen et al. 1999] introduced hierarchical temporal organization for time-varying volume rendering. Our work follows the same broad goal of exploiting temporal coherence, but with a renderer-facing asset contract: a single packed representation should remain resident and answer direct samples in both space and time. Dense scientific fields are evaluated as stress tests for this query contract, whereas the primary target is animated smoke/fire asset storage and rendering.

3 Method

3.1 Overview

Figure 2 summarizes the overall system. The central design goal is to support direct renderer and query access from the compressed representation, without dense-frame materialization or a sequential scan. This requires that every block be independently decodable, that its location be directly addressable, and that its decode cost be bounded regardless of content. The input 4D scalar volume is partitioned into fixed $8 \times 8 \times 8$ spatial leaves. Rendering-oriented blocks use a temporal-first field-adaptive encoder that preserves visible smoke/fire events before packed residual organization. Auxiliary scientific blocks use a spatial-first field-adaptive encoder: a shared coarse model is constructed first, then the residual is routed to one of several block modes for dense query stress tests. Both routes write into the same VBTPACK4 layout: a global block offset table and a shared payload pool. Query follows one stable path: locate block via offset table, read the self-descriptive header, decode the coarse component, and dispatch to the mode-specific residual decode.

3.2 Temporal-first Rendering Encoder

The main VBT encoder targets animated smoke/fire assets. Unlike dense scientific fields, sparse rendering volumes contain large empty regions and visually important density concentrated near moving boundaries and short-lived temporal events. A spatial-first encoder may achieve low average error while smearing density birth, extinction, or boundary motion across time. VBT therefore uses a temporal-first rendering encoder: it first detects voxel-level temporal transitions and then chooses a compact temporal grid representation for each 8^3 leaf.

After empty-block rejection, the encoder selects among three active render modes. *Empty* stores no payload. *TemporalGrid4* stores a 4^3 temporal control grid with an FP16 keyframe codec and is used for smooth or low-visibility blocks. *TemporalFine6* stores a 6^3 temporal grid and is selected when coarse temporal reconstruction would smear visible density transitions. The selected render mode is written as a mode-tagged VBTPACK4 block, so the same compressed asset can be sampled by both standalone queries and the native Cycles path. The packed tag id for TemporalFine6 remains 3 for compatibility with the current layout; tag 2 is reserved outside the main evaluation.

3.3 Packed Backend and Native Cycles Sampling

All render and auxiliary stress-test encoders write into the same VBTPACK4 layout: a file header, a global offset table, and a payload pool. The offset table gives $O(1)$ block lookup, while each block header stores a compact mode tag and mode-specific decode parameters. Here $O(1)$ refers to block location rather than identical per-sample latency, because decode work remains mode-dependent. The decode constant is bounded, but dense grids, sparse events, and empty/coarse blocks perform different local work.

At render time, the native Cycles prototype invokes the VBT sampler directly from the volume sampling path. A sample at (x, y, z, t) maps to a leaf, loads the packed header, evaluates the temporal representation, and dispatches to the mode-specific render decode. This path does not materialize dense frames and does not convert the asset to an OpenVDB proxy. The same layout also supports standalone GPU query kernels and CPU exact queries, but renderer sampling is the primary runtime target. The GPU query path is implemented as a single-dispatch compute kernel with an AoS query layout, following GPU-based volume rendering principles [Aila and Laine 2009; Krüger and Westermann 2003; Westermann and Ertl 1998].

3.4 Auxiliary Dense-field Stress-test Encoder

For the JHTDB stress tests, we use a separate spatial-first encoder that maps dense turbulence fields into the same VBTPACK4 backend. This route is not the primary rendering contribution; it is used to test whether the packed layout remains queryable on dense physical fields with local high-error excursions. The encoder builds a GRID4 coarse base: each 8^3 block is approximated by a 4^3 control grid, and the temporal evolution of each control point is compressed using a truncated Type-II DCT [Ahmed et al. 1974]. It then routes residuals to a small capacity ladder: Coarse Only, Sparse Events, DenseGrid3, and gated DenseGrid4. Mode selection uses a local tail-aware score, $D(\mathbf{e}) = \max(\text{RMSE}(\mathbf{e}), w_{99}P_{99}(\mathbf{e}), w_{\text{peak}}\|\mathbf{e}\|_{\infty})$, with fixed weights used only for the auxiliary dense-field route. Detailed parameters for the byte-rate term, upgrade gate, and quantization settings are provided in the supplementary material.

4 Experiments

4.1 Setup

We evaluate VBT primarily as a compressed animated-volume asset backend on four EmberGen free VDB sequences [JangaFX 2023]: three smoke-style volumes and one fire/explosion sequence. Rendering quality is measured under fixed Blender/Cycles settings, while the native integration test uses a modified Blender/Cycles build that loads .vbtp volumes directly. All GPU experiments were run on an NVIDIA GeForce RTX 5070 Laptop GPU with 8 GB of dedicated VRAM. As auxiliary dense-field stress tests, we also evaluate eight FP32 time-varying cutouts (J1–J8) from the Johns Hopkins Turbulence Databases (JHTDB) [Li et al. 2008], spanning five source datasets: Forced Isotropic Turbulence (J1: pressure, J2: u_x), Homogeneous Buoyancy Driven Turbulence (J3: density, J4: pressure), Forced MHD Turbulence (J5: b_x), Channel Flow (J6: pressure, J7: u_x), and Transitional Boundary Layer (J8: pressure). J1–J7 use $256^3 \times 128$ cutouts (8.0 GiB raw); J8 uses $256 \times 224 \times 256 \times 128$ (7.0 GiB raw).

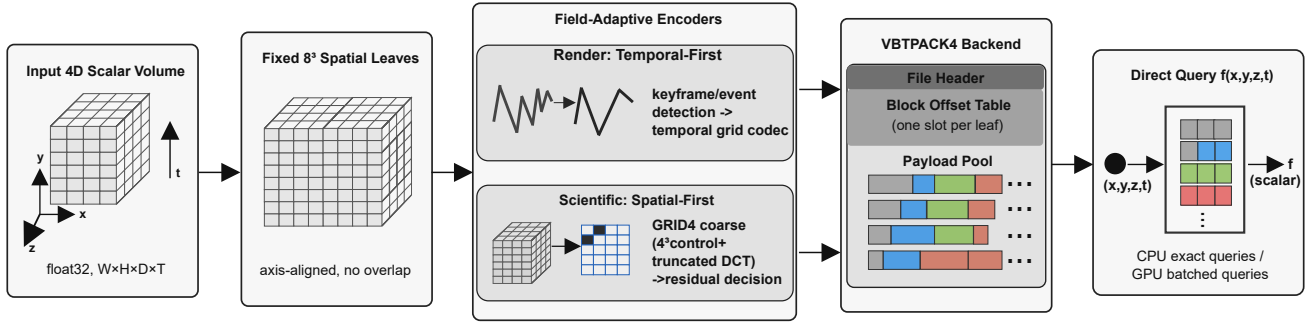


Fig. 2. System overview. A 4D volume is partitioned into fixed 8^3 spatial leaves and encoded by one of two field-adaptive encoders. The rendering route applies a temporal-first keyframe-aware encoding for animated smoke/fire assets; the auxiliary scientific route applies a spatial-first coarse model followed by residual routing for dense query stress tests. Both write into the same VBTPACK4 packed backend with a global block offset table and a shared payload pool, enabling $O(1)$ block lookup and a unified direct query path on CPU and GPU.

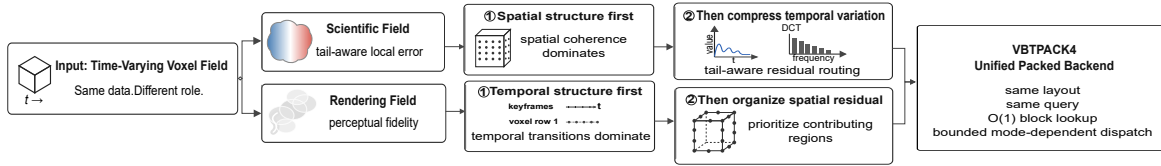


Fig. 3. Field-adaptive encoder rationale. The primary smoke/fire route is temporal-first because visible density structure is concentrated at moving boundaries and time-local transitions. The auxiliary dense-field route is spatial-first because within-frame coherence dominates in the JHTDB stress tests. Both encoders write into the same VBTPACK4 backend.

The resident-4D query benchmark in Table 5 uses CUDA kernels for both VBT and zfp, so the reported throughput comparison is same-API rather than Vulkan-versus-CUDA.

4.2 Native Cycles Compressed-Domain Rendering

We implemented a native Cycles prototype that loads .vbtp assets directly as Blender volume data and invokes the VBT sampler from both CPU and CUDA rendering paths. The render path does not convert the asset to OpenVDB. We validate this path using `industrial_chimney_mainline.vbtp` on an NVIDIA GeForce RTX 5070 Laptop GPU with CUDA enabled. Figure 1 provides the corresponding rendering evidence, and Table 1 summarizes the validated capabilities without relying on proxy conversion. Build and render logs confirming direct .vbtp loading and CUDA VBT sampler compilation are included in the supplementary material.

This is a compressed-domain rendering proof for the resident asset contract rather than a full production Cycles backend or broad renderer benchmark. The native path is important because the renderer samples the compressed asset directly at shading time. Unlike a proxy workflow, the .vbtp object is not expanded to an OpenVDB volume before rendering; the sampling path resolves the block and mode tag inside the Cycles volume evaluation. This validates VBT as a renderer-facing asset representation rather than only an offline compression format.

Table 1. Native Cycles prototype validation. The native path renders a resident .vbtp asset directly and does not use OpenVDB proxy conversion.

Capability	Validation
Native asset loading	.vbtp volume datablock loaded directly
OpenVDB proxy bypass	no proxy conversion in render path
CPU Cycles path	completed native .vbtp render
CUDA Cycles path	completed native .vbtp render
Device	RTX 5070 Laptop GPU
Device sampler	VBT sampler compiled into CUDA path

4.3 Rendering Quality and Storage

Rendering is the primary graphics-facing runtime use case: the temporal-first route turns VDB sequences into resident compressed 4D assets for direct density sampling. With identical Blender/Cycles settings, the three smoke-style cases remain consistently strong, while `ground_explosion` is the clearly harder transient sequence. Table 2 summarizes source-VDB footprint, framewise rendering quality, and Fine6 usage in the final packed assets. Across the four smoke/fire sequences, VBT is 25.9–60.3× smaller than the dense RAW sequences and about 1.8–5.1× smaller than the corresponding source OpenVDB assets. The chimney, tornado, and dust-devil cases remain stable, whereas the explosion case is the clearly harder transient case and reaches the lowest worst-frame

Table 2. Rendering-oriented sequence summary under fixed Blender/Cycles settings. Worst reports the lowest framewise metric over the sequence. Fine6 is the fraction of leaves routed to the highest-capacity temporal render mode.

Sequence	Frames	VDB	VBT	VDB→VBT	Mean PSNR	Worst PSNR	Mean SSIM	Fine6
industrial_chimney	250	2.61 GB	514.1 MB	5.1×	41.69	37.94	0.9950	8.7%
tornado_looping	128	2.17 GB	937.9 MB	2.3×	47.33	45.73	0.9960	5.7%
dust_devil	101	0.57 GB	313.8 MB	1.8×	43.08	32.49	0.9914	9.1%
ground_explosion	200	1.24 GB	399.4 MB	3.1×	47.04	24.41	0.9536	35.1%

Table 3. Fixed smoke ray-marching benchmark on industrial_chimney (frames 96–101, 640×360 rays, 192 steps). Dense and NanoVDB use a three-frame cache; VBT keeps the full temporal stream resident. Times are in ms.

Backend	Kernel	Upload	Total
dense	0.481	135.023	135.504
NanoVDB	3.241	18.586	21.827
VBT	8.288	0.000	8.288

quality at 24.41 dB PSNR and 0.7758 SSIM. Figure 5 shows all four EmberGen sequences in rendered form. The harder ground_explosion sequence routes 35.1% of leaves to Fine6, reflecting the larger fraction of high-visibility transient blocks. Smoke-plume sequences remain dominated by Empty and Grid4 leaves, and no reserved mode tags appear in the evaluated render assets. Full 5%-tail PSNR/SSIM values are reported in the supplementary material.

4.4 Runtime Sampling and Ray Marching

Table 3 reports a fixed smoke ray-marching benchmark on the chimney sequence. Dense and NanoVDB have faster sampling kernels for this narrow primitive, but they require per-frame advance uploads under the tested three-frame cache policy. VBT keeps the full stream resident: its kernel time is 8.288 ms with no upload, while NanoVDB totals 21.827 ms and dense totals 135.504 ms once upload cost is included. This benchmark is not a complete renderer evaluation; it isolates the resident-stream advantage of the packed 4D asset under a fixed ray-marching workload.

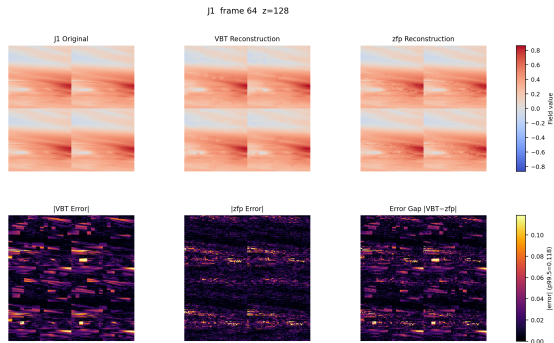


Fig. 4. J1 frame-64 slice comparison. Original, VBT, and zfp reconstructions are shown with error maps, providing a slice-level view of the local-error behavior summarized in Table 4.

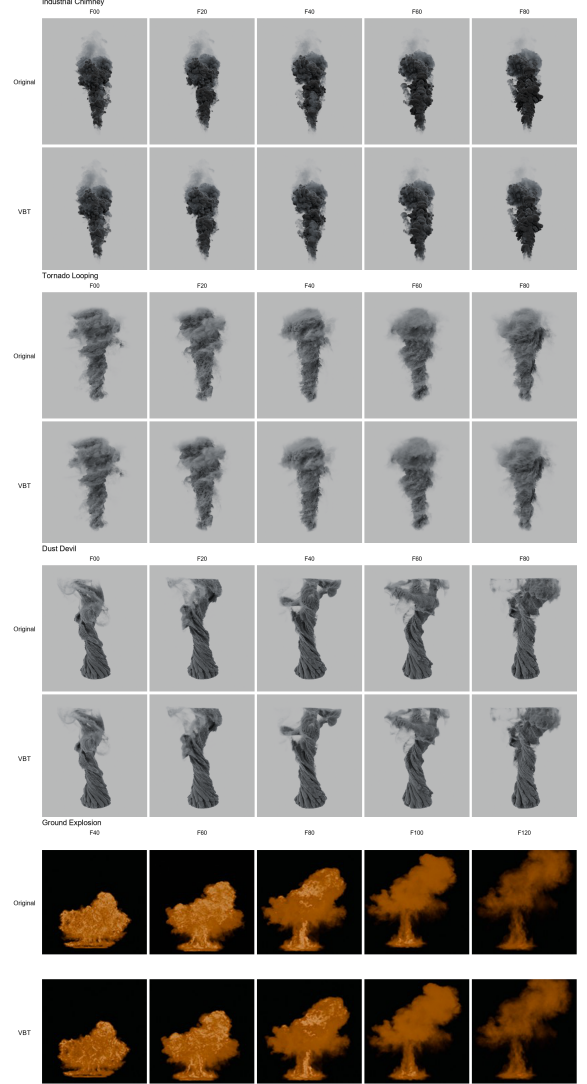


Fig. 5. Qualitative rendering comparisons across all four EmberGen sequences. Each mini-panel shows original Blender renders above VBT reconstructions under the same fixed camera and rendering settings.

4.5 Auxiliary Dense-Field Stress Test

Scientific fields serve as auxiliary dense physical-query stress tests for the resident packed backend, not as evidence that VBT is an RMSE-optimal scientific compressor or Pareto-dominant scientific

Table 4. Auxiliary dense-field stress-test metrics for the resident backend and near-size zfp operating points on J1–J8. These results are matched operating-point comparisons, not scientific-compression Pareto frontiers. Ratio is raw size divided by compressed size. J1–J8 are JHTDB cutouts: J1 FIT pressure, J2 FIT u_x , J3 HBDT density, J4 HBDT pressure, J5 MHD b_x , J6 Channel pressure, J7 Channel u_x , J8 TBL pressure.

Dataset	VBT				zfp			
	Size (MB)	Ratio	RMSE	MaxAbs	Size (MB)	Ratio	RMSE	MaxAbs
J1	81.8	100×	0.0346	0.848	67.1	122×	0.0230	2.930
J2	139.9	58×	0.0494	1.171	134.2	61×	0.0440	1.457
J3	112.0	73×	8.44e-4	0.016	134.2	61×	5.99e-4	0.037
J4	78.5	104×	9.3e-5	0.004	67.1	122×	1.11e-4	0.010
J5	128.8	64×	0.0364	0.498	134.2	61×	0.0279	1.489
J6	101.4	81×	4.19e-4	0.012	134.2	61×	3.26e-4	0.030
J7	108.5	76×	0.0052	0.109	134.2	61×	0.0059	0.239
J8	107.7	67×	0.0030	0.015	117.4	61×	0.0024	0.087

codec. Table 4 reports matched operating-point metrics against fixed-rate zfp [Lindstrom 2014]. At these points, VBT reaches 58–104× compression and lowers MaxAbs on all eight cutouts, while zfp usually gives lower RMSE. This supports the intended tail-aware local behavior of the packed backend under dense direct queries, rather than broad dominance over scientific codecs. Figures 4, 7, and 9 visualize this behavior through slice comparisons, axis probes, error CDFs, and temporal traces.

We also compare against SZ3 [Di and Cappello 2016; Liang et al. 2023] in the supplementary material. As expected for an error-bounded scientific codec, SZ3 gives lower RMSE and MaxAbs than VBT in this offline reconstruction setting. This defines the boundary of the contribution: VBT targets resident direct (x, y, z, t) queries and native renderer sampling from one compressed animated-volume asset, not replacement of offline scientific compressors.

Table 5 reports a resident random 4D query benchmark against fixed-rate zfp. Both backends upload the full compressed sequence once and answer the same random query list on the GPU. VBT sustains 4.60×10^8 – 1.28×10^9 queries/s after locality-aware sorting, while zfp remains faster on the narrower frame-local block-decode primitive. Thus, the benchmark should be read as a backend stress test rather than a fastest-query-codec claim. These dense-field experiments remove the favorable assumptions present in sparse smoke/fire assets. The JHTDB cutouts have no empty-space structure to exploit, no transfer-function masking, and no visually irrelevant background regions; any location may be queried as a physically meaningful value. They therefore test whether the same VBTPACK4 layout remains usable when the workload changes from renderer samples on sparse animated assets to dense probe-style queries over continuous fields. The result is not that VBT becomes a superior scientific compressor, but that the resident packed representation remains functional outside the sparse-volume regime.

Table 5. Resident 4D CUDA query benchmark. Values are VBT / zfp in Gq/s.

Dataset	Resident MB	Unsorted	Sorted
J1	81.79 / 67.11	0.72 / 2.57	1.28 / 2.28
J2	139.89 / 134.22	0.38 / 1.21	0.55 / 1.38
J7	108.45 / 134.22	0.47 / 1.35	0.73 / 1.26
J8	107.69 / 117.44	0.32 / 1.30	0.46 / 1.36

4.6 Auxiliary Dense-field Ablation

This ablation concerns only the auxiliary dense-field encoder used in Section 4.5. Table 6 studies the mode decision stack on J2 and J7. Starting from DenseGrid3 plus stratified sparse events, ungated DenseGrid4 lowers RMSE but at unacceptable size increase and over-upgrades most leaves (J7: 90.4 MB $\rightarrow$ 164.0 MB, 97% DenseGrid4). The DenseGrid4 gate keeps most of the RMSE gain while sharply reducing size and DenseGrid4 use (J7: 108.5 MB, 24% DenseGrid4). Adaptive coarse keep provides a modest additional size reduction on smooth fields and has no effect when the selected keep class is already optimal.

Table 6. Auxiliary dense-field router ablation. This table concerns only the stress-test encoder, not the temporal-first rendering encoder. Size/error cells report MB / full-volume RMSE. DG4% is the fraction of leaves routed to DenseGrid4.

Variant	J2 MB/RMSE	J2 DG4%	J7 MB/RMSE	J7 DG4%
DG3 + sparse	90.4 / 0.0617	0	90.4 / 0.00608	0
+ DG4 ungated	156.7 / 0.0475	82	164.0 / 0.00407	97
+ DG4 gated	139.9 / 0.0494	60	108.5 / 0.00525	24

4.7 Renderer-side Implications

The native Cycles prototype changes the role of VBT from an offline compressed cache into a renderer-facing volume asset. In a conventional per-frame VDB workflow, temporal coherence is external to the renderer: frames are loaded, cached, or converted independently, and time-varying sampling requires a policy for keeping nearby

frames resident. VBT instead exposes the whole sequence as one packed .vbtp object. A Cycles volume sample can resolve (x, y, z, t) directly through the block offset table and mode tag, without first materializing dense frames or converting to an OpenVDB proxy. Unlike OpenVDB and NanoVDB, which operate as per-frame sparse volume caches, VBT is not merely a smaller cache: it stores the sequence as a single compressed .vbtp asset that can be sampled directly by the renderer.

This design does not make the current prototype a production-complete Cycles backend, and Table 3 shows that the VBT sampling primitive is slower than dense or NanoVDB sampling in isolation. The advantage is the resident-stream contract: the full animated asset can remain in compressed form, and frame advance does not require uploading a new dense or NanoVDB frame cache. This is the setting targeted by VBT: long smoke/fire sequences that need compact storage, direct renderer sampling, and occasional probe queries from the same representation.

Operationally, this changes the unit of residency. In a per-frame VDB or NanoVDB workflow, the renderer or host application must decide which frames are resident, when to upload or evict them, and how to handle time samples that fall between cached frames. With VBT, the residency unit is the compressed sequence itself: all time samples resolve through one offset table and one mode-dispatch path. This does not remove all scheduling concerns—ray ordering, cache locality, material-system integration, and viewport interaction remain future work—but it gives the renderer a simpler contract: the animated volume can remain compressed, resident, and directly sampleable across the frame range.

This contract is also useful for authoring and inspection workflows. A DCC tool often needs more than final-frame rendering: artists scrub time, inspect density at selected locations, compare frames, and preview temporal changes before committing to a full render. In a per-frame cache, these operations may exercise different data paths, for example viewport proxies, frame-local GPU caches, or offline decompressed arrays. VBT instead exposes the same resident asset to renderer samples and probe-style queries. This does not make VBT a complete DCC system, but it clarifies the intended abstraction: the compressed 4D asset is the runtime object, not only an interchange file or an offline archive.

5 Discussion and Limitations

VBT should be read as a compressed animated-volume asset backend, not as the best compressor on every scalar metric. Its value is that ray samples, inspector probes, and scalar-history queries can use one resident representation instead of switching among per-frame VDB caches, dense arrays, and offline codec streams. Fixed-rate zfp remains faster for the narrower block-decode query and often lower in RMSE; VBT instead couples direct 4D semantics, field-adaptive local models, and tail-aware local-error behavior in one packed backend. A 4D backend also does not automatically improve every compression metric: our scientific stress-test route shares temporal structure mainly in the coarse layer, while residuals are still mostly spatial.

This distinction matters because a smaller per-frame cache and a compressed 4D asset expose different programming models. A per-frame representation asks the renderer to manage time externally: frame selection, interpolation policy, residency, and upload scheduling are handled outside the asset. A VBT asset internalizes the temporal dimension into the sampled representation itself. The renderer still performs ordinary volume integration, but the density lookup is no longer tied to a materialized frame. This is the main systems claim of VBT: temporal compression, storage reduction, and direct sampling are expressed through one addressable runtime layout.

Limitations and future work. The native Cycles integration is currently a prototype rather than a production-complete Blender feature. We validate direct .vbtp rendering on CPU and CUDA paths, but do not yet provide a full interactive viewport workflow, production-scale scheduling optimization, or broad renderer benchmark. Scientific results are auxiliary stress tests; we do not provide full rate-distortion frontiers over zfp, SZ3, and MGARD, and J5/J8 remain hard dense-field cases. Encoding is offline; future work targets stronger hard-case distortion proxies, richer residuals, a full production renderer/plugin workflow, and larger interactive DCC stress tests.

6 Conclusion

We presented VBT, a queryable compressed 4D backend for animated volume assets. Across smoke/fire rendering sequences, it stores animated volumes as resident .vbtp assets and can be sampled directly by a native Cycles prototype without OpenVDB proxy conversion. Dense scientific fields serve as stress tests for the same queryable backend, where VBT reaches 58–104× compression at matched zfp operating points and sustains 4.6×10^8 – 1.28×10^9 resident 4D queries/sec after locality-aware sorting. The main result is therefore a renderer-facing compressed 4D asset contract: an animated volume sequence can remain compact, resident, and directly sampleable throughout the native rendering path.

References

- Nasir Ahmed, T. Natarajan, and Kamisetty Ramamohan Rao. 1974. Discrete Cosine Transform. *IEEE Trans. Computers* 23, 1 (1974), 90–93. doi:10.1109/T-C.1974.223784
- Timo Aila and Samuli Laine. 2009. Understanding the efficiency of ray traversal on GPUs. In *Proceedings of the ACM SIGGRAPH/EUROGRAPHICS Conference on High Performance Graphics 2009, New Orleans, Louisiana, USA, August 1-3, 2009*, Stephen N. Spencer, David K. McAllister, Matt Pharr, Ingo Wald, David P. Luebke, and Philipp Slusallek (Eds.). Eurographics Association, 145–149. doi:10.2312/EGGH/HPG09/145-150
- Rafael Ballester-Ripoll, Peter Lindstrom, and Renato Pajarola. 2020. TTHRESH: Tensor Compression for Multidimensional Visual Data. *IEEE Trans. Vis. Comput. Graph.* 26, 9 (2020), 2891–2903. doi:10.1109/TVCG.2019.2904063
- Rafael Ballester-Ripoll and Renato Pajarola. 2016. Lossy volume compression using Tucker truncation and thresholding. *Vis. Comput.* 32, 11 (2016), 1433–1446. doi:10.1007/S00371-015-1130-Y
- Cyril Crassin, Fabrice Neyret, Sylvain Lefebvre, and Elmar Eisemann. 2009. GigaVoxels: ray-guided streaming for efficient and detailed voxel rendering. In *Proceedings of the 2009 Symposium on Interactive 3D Graphics, SI3D 2009, February 27 - March 1, 2009, Boston, Massachusetts, USA*, Eric Haines, Morgan McGuire, Daniel G. Aliaga, Manuel M. Oliveira, and Stephen N. Spencer (Eds.). ACM, 15–22. doi:10.1145/1507149.1507152
- Sheng Di and Franck Cappelto. 2016. Fast Error-Bounded Lossy HPC Data Compression with SZ. In *30th IEEE International Parallel and Distributed Processing Symposium, IPDPS 2016*, IEEE, 730–739. doi:10.1109/IPDPS.2016.11
- Robert A. Drebin, Loren C. Carpenter, and Pat Hanrahan. 1988. Volume rendering. In *Proceedings of the 15th Annual Conference on Computer Graphics and Interactive Techniques, SIGGRAPH 1988, Atlanta, Georgia, USA, August 1-5, 1988*, Richard J. Beach (Ed.). ACM, 65–74. doi:10.1145/54852.378484
- Qian Gong, Jieyang Chen, Ben Whitney, Xin Liang, Viktor Reshniak, Tania Banerjee, Jaemoon Lee, Anand Rangarajan, Lipeng Wan, Nicolas Vidal, Qing Liu, Ana Gainaru, Norbert Podhorski, Richard Archibald, Sanjay Ranka, and Scott Klasky. 2024. MGARD: A multigrid framework for high-performance, error-controlled data compression and refactoring. *CoRR abs/2401.05994* (2024). arXiv:2401.05994 doi:10.48550/ARXIV.2401.05994
- Duong Hoang, Pavol Klacansky, Harsh Bhatia, Peer-Timo Bremer, Peter Lindstrom, and Valerio Pascucci. 2021. Efficient and Flexible Hierarchical Data Layouts for a Unified Encoding of Scalar Field Precision and Resolution. *IEEE Trans. Vis. Comput. Graph.* 27, 2 (2021), 603–613. doi:10.1109/TVCG.2020.3030381
- Rama Karl Hoetzlein. 2016. GVDB: raytracing sparse voxel database structures on the GPU. In *Proceedings of High Performance Graphics, HPG 2016, Dublin, Ireland, June 20-22, 2016*, David Luebke and Steven Molnar (Eds.). Eurographics Association, 109–117. doi:10.2312/HPG.20161197
- Insung Ihm and Sanghun Park. 1999. Wavelet-Based 3D Compression Scheme for Interactive Visualization of Very Large Volume Data. *Comput. Graph. Forum* 18, 1 (1999), 3–15. doi:10.1111/1467-8659.00298
- JangaFX. 2023. Free VDB Animations — EmberGen Sample Library. <https://jangafx.com/software/embergen/download/free-vdb-animations>. CC0 Public Domain.
- Jens H. Krüger and Rüdiger Westermann. 2003. Acceleration Techniques for GPU-based Volume Rendering. In *14th IEEE Visualization Conference, IEEE Vis 2003, Seattle, WA, USA, October 19-24, 2003, Proceedings*, Greg Turk, Jarke J. van Wijk, and Robert J. Moorhead II (Eds.). IEEE Computer Society, 293–300. doi:10.1109/VISUAL.2003.1250385
- Han-Wei Shen, Ling-Jan Chiang, and Kwan-Liu Ma. 1999. A Fast Volume Rendering Algorithm for Time-Varying Fields Using a Time-Space Partitioning (TSP) Tree. In *10th IEEE Visualization Conference, IEEE Vis 1999, San Francisco, CA, USA, October 24-29, 1999, Proceedings*, David S. Ebert, Markus H. Gross, and Bernd Hamann (Eds.). IEEE Computer Society and ACM, 371–377. doi:10.1109/VISUAL.1999.809910
- Rüdiger Westermann and Thomas Ertl. 1998. Efficiently Using Graphics Hardware in Volume Rendering Applications. In *Proceedings of SIGGRAPH 1998*. ACM, 169–177. doi:10.1145/280814.280860
- Jinyang Wu, Sheng Di, Xin Liang, Kai Zhao, Zizhong Chen, and Franck Cappelto. 2023. cuSZ-i: High-ratio scientific lossy compression on GPUs with optimized multi-level interpolation. *IEEE International Parallel and Distributed Processing Symposium* (2023), 745–756. doi:10.1109/IPDPS54959.2023.00081
- Xin Liang, Sheng Di, Dingwen Tao, Sihuan Li, Shaomeng Li, Hanqi Guo, Zizhong Chen, and Franck Cappelto. 2018. Error-Controlled Lossy Compression Optimized for High Compression Ratios of Scientific Datasets. In *IEEE International Conference on Big Data*. IEEE, 438–447. doi:10.1109/BigData.2018.8622520
- Xin Liang, Ben Whitney, Jieyang Chen, Lipeng Wan, Qing Liu, Dingwen Tao, James Kress, David Pugmire, Matthew Wolf, Norbert Podhorski, and Scott Klasky. 2022. MGARD+: Optimizing Multilevel Methods for Error-Bounded Scientific Data Reduction. *IEEE Trans. Computers* 71, 7 (2022), 1522–1536. doi:10.1109/TC.2021.3092201
- Xin Liang, Kai Zhao, Sheng Di, Sihuan Li, Robert Underwood, Ali Murat Gok, Jiannan Tian, Junjing Deng, Jon C. Calhoun, Dingwen Tao, Zizhong Chen, and Franck Cappelto. 2023. SZ3: A Modular Framework for Composing Prediction-Based Error-Bounded Lossy Compressors. *IEEE Trans. Big Data* 9, 2 (2023), 485–498. doi:10.1109/TBData.2022.3201176
- Peter Lindstrom. 2014. Fixed-Rate Compressed Floating-Point Arrays. *IEEE Trans. Vis. Comput. Graph.* 20, 12 (2014), 2674–2683. doi:10.1109/TVCG.2014.2346458
- Peter Lindstrom, Duong Hoang, Sheng Di, Franck Cappelto, Mark Ainsworth, and John Googler Burstedde. 2025. Survey of Lossy Compression Techniques for Scientific Data. *Int. J. High Perform. Comput. Appl.* 39, 1 (2025), 104–122. doi:10.1177/10943420241284023
- Peter Lindstrom and Martin Isenburg. 2006. Fast and Efficient Compression of Floating-Point Data. *IEEE Trans. Vis. Comput. Graph.* 12, 5 (2006), 1245–1250. doi:10.1109/TVCG.2006.143
- Yuzhe Lu, Kairong Jiang, Joshua A. Levine, and Matthew Berger. 2021. Compressive Neural Representations of Volumetric Scalar Fields. *Comput. Graph. Forum* 40, 3 (2021), 135–146. doi:10.1111/CGF.14295
- Nelson L. Max. 1995. Optical Models for Direct Volume Rendering. *IEEE Trans. Vis. Comput. Graph.* 1, 2 (1995), 99–108. doi:10.1109/2945.468400
- Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. 2020. NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis. In *Computer Vision - ECCV 2020 - 16th European Conference, Glasgow, UK, August 23-28, 2020, Proceedings, Part I (Lecture Notes in Computer Science)*, Andrea Vedaldi, Horst Bischof, Thomas Brox, and Jan-Michael Frahm (Eds.). Springer, 405–421. doi:10.1007/978-3-030-58452-8_24
- Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. 2022. Instant neural graphics primitives with a multiresolution hash encoding. *ACM Trans. Graph.* 41, 4 (2022), 102:1–102:15. doi:10.1145/3528223.3530127
- Ken Museth. 2013. VDB: High-resolution sparse volumes with dynamic topology. *ACM Trans. Graph.* 32, 3 (2013), 27:1–27:22. doi:10.1145/2487228.2487235
- Ken Museth. 2021. NanoVDB: A GPU-Friendly and Portable VDB Data Structure For Real-Time Rendering And Simulation. In *SIGGRAPH 2021 Talks*. ACM, 1:1–1:2. doi:10.1145/3450623.3464653
- Miguel Balsa Rodríguez, Enrico Gobbetti, Jose Antonio Iglesias Guitian, Maxim Makhinya, Fabio Marton, Renato Pajarola, and Susanne K. Suter. 2014. State of the Art in Compressed GPU-Based Direct Volume Rendering. *Computer Graphics Forum* 33, 6 (2014), 77–100. doi:10.1111/cgf.12280
- Jens Schneider and Rüdiger Westermann. 2003. Compression Domain Volume Rendering. In *14th IEEE Visualization Conference, IEEE Vis 2003, Seattle, WA, USA, October 19-24, 2003, Proceedings*, Greg Turk, Jarke J. van Wijk, and Robert J. Moorhead II (Eds.). IEEE Computer Society, 293–300. doi:10.1109/VISUAL.2003.1250385
- Han-Wei Shen, Ling-Jan Chiang, and Kwan-Liu Ma. 1999. A Fast Volume Rendering Algorithm for Time-Varying Fields Using a Time-Space Partitioning (TSP) Tree. In *10th IEEE Visualization Conference, IEEE Vis 1999, San Francisco, CA, USA, October 24-29, 1999, Proceedings*, David S. Ebert, Markus H. Gross, and Bernd Hamann (Eds.). IEEE Computer Society and ACM, 371–377. doi:10.1109/VISUAL.1999.809910
- Rüdiger Westermann and Thomas Ertl. 1998. Efficiently Using Graphics Hardware in Volume Rendering Applications. In *Proceedings of SIGGRAPH 1998*. ACM, 169–177. doi:10.1145/280814.280860
- Jinyang Wu, Sheng Di, Xin Liang, Kai Zhao, Zizhong Chen, and Franck Cappelto. 2023. cuSZ-i: High-ratio scientific lossy compression on GPUs with optimized multi-level interpolation. *IEEE International Parallel and Distributed Processing Symposium* (2023), 745–756. doi:10.1109/IPDPS54959.2023.00081

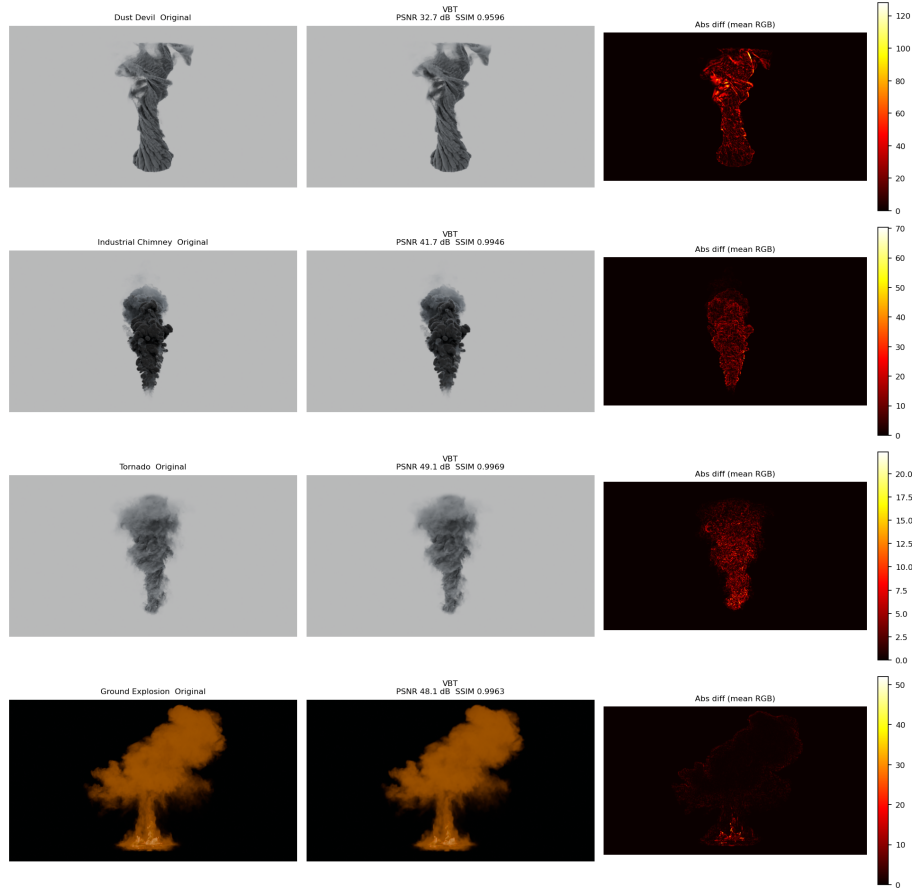


Fig. 6. Representative rendered frame comparisons for all four smoke/fire sequences. Each row shows the original render, the VBT reconstruction, and the absolute RGB difference map under the same Blender/Cycles camera and lighting setup.

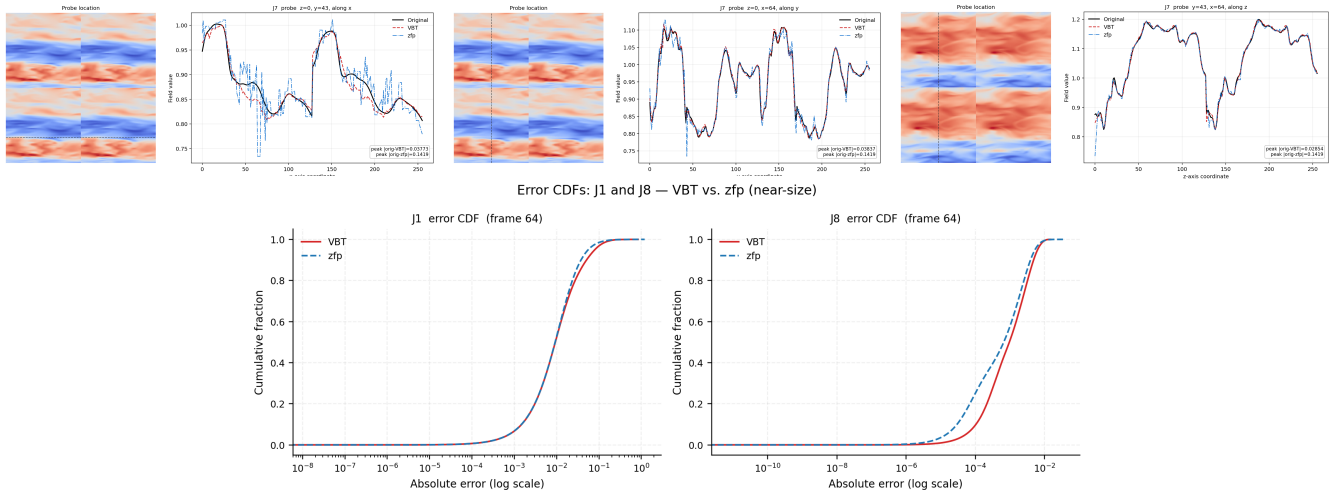


Fig. 7. Additional dense-field probes. Top: J7 frame-64 axis-aligned line probes at representative hard locations. Bottom: paired J1/J8 error CDFs showing fewer large-error voxels at the selected operating points.

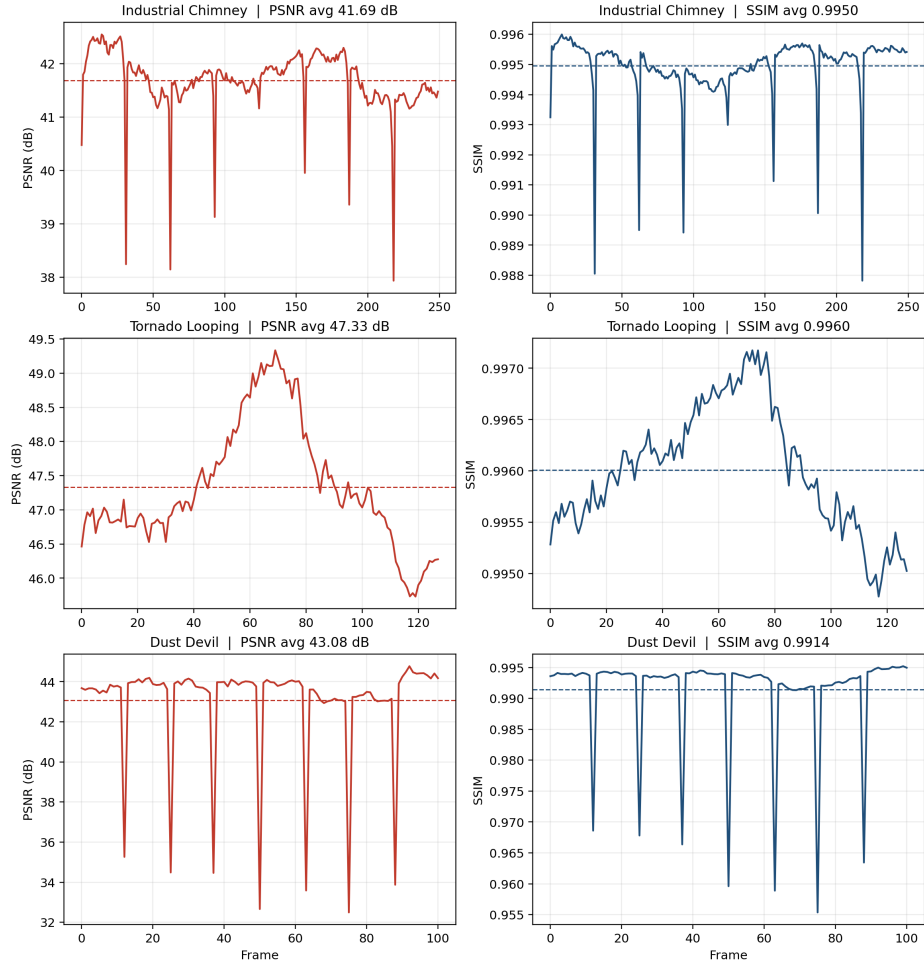


Fig. 8. Per-frame rendering quality under fixed Blender/Cycles settings.

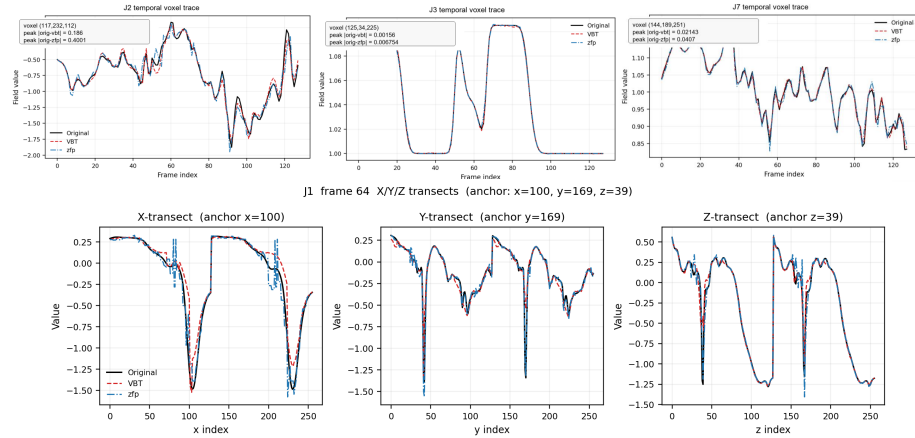


Fig. 9. Representative local scientific probes beyond aggregate RMSE.